Recursive Analysis, Substitution & Recurrence Trees

From recursive code → recurrence relation → repeated expansion → visual cost tree → final Θ growth.

LECTURE 9/10

$T(n)$

↓

smaller calls

↓

pattern

↓

$\Theta(\cdot)$

What should you be able to do by the end?

Two lectures, one continuous story: model recursive work first, then solve it in two different ways.

MODEL

Write T(n) from a recursive algorithm.

EXPAND

Unroll a recurrence until a pattern appears.

VERIFY

Use substitution to justify a proposed bound.

TREE

Build a recurrence tree and calculate level costs.

INTERPRET

State the final Θ class in plain English.

Main running example: Merge Sort — first understand the algorithm, then derive and solve its recurrence.

Where Lecture 9/10 fits

Lecture 8 finished non-recursive comparison. Now the algorithm itself calls smaller copies of itself.



Lecture 11 will be easier because the Master Theorem will summarize patterns you already understand.

The story of these two lectures

Do not start by memorizing formulas. Start by asking what one recursive call actually does.

1 — TRACE

Run a tiny input and watch the recursive calls.

2 — MODEL

Turn the call structure into T(n).

3 — SOLVE

Expand or draw a tree until the total work is visible.

4 — CLASSIFY

Convert the result into Θ(·) and explain it.

Concrete input → pattern → general recurrence → asymptotic conclusion.

PART A — LECTURE 9

From recursive code to a recurrence

A recurrence is simply a mathematical description of how the current call depends on smaller calls.

Build understanding from a concrete trace before generalizing the mathematics.

Recall: what makes an algorithm recursive?

Lecture 5 focused on correctness. Today we use the same call structure to study efficiency.

BASE CASE

A smallest input is solved directly.
No further recursive call.

RECURSIVE CASE

The current problem is reduced to one or more smaller instances.

PROGRESS

Each recursive call moves toward the base case.

Correctness asked “does it return the right answer?” Analysis asks “how much total work do all calls perform?”

What changes when the algorithm calls itself?

For a loop, we count repeated operations directly. For recursion, the repeated work is hidden inside smaller calls.

NON-RECURSIVE

SUM(A)

One loop processes n elements.
We can count the additions directly.

$C(n) = n$

RECURSIVE

COUNT(n)

One call performs a small amount of work and then asks COUNT(n−1) to finish the smaller problem.

We need T(n) in terms of T(n−1).

A recurrence exposes the work hidden inside recursive calls.

What does T(n) mean here?

T(n) represents the running-time model for an input of size n.

$$T(n) = \text{work done by the current call} + \text{work done by recursive call(s)}$$

CURRENT CALL

comparisons • arithmetic • splitting • merging • other non-recursive work

SMALLER CALLS

$T(n-1)$, $T(n/2)$, $2T(n/2)$, ...

BASE CASE

Usually $T(1)=\Theta(1)$ or $T(0)=\Theta(1)$.

Do not confuse this T(n) with the exact basic-operation count C(n) from Lecture 6.

A recurrence has a few pieces

Identify these pieces before trying to solve anything.

INPUT SIZE

What does n measure?

OF CALLS

How many recursive calls are created?

CALL SIZE

n−1? n/2? n/3?

OUTSIDE WORK

What does the current call do besides recursion?

BASE CASE

Where does recursion stop?

First derive the recurrence correctly. Solving the wrong recurrence perfectly is still wrong.

Worked example 1: COUNTDOWN

Start with a recursive algorithm whose call structure is obvious.

PSEUDOCODE

```
COUNTDOWN(n)
    if n = 1
        return
    print(n)
    COUNTDOWN(n - 1)
```

ONE SMALLER CALL

COUNTDOWN(n−1)

CURRENT WORK

one print + constant overhead

QUESTION

How many calls happen before n reaches 1?

Use a tiny n first. Then generalize.

1
1
1

Trace COUNTDOWN(5)

Watch the input shrink by exactly 1 at each recursive step.



Each arrow represents one recursive call on a smaller input.

$$T(5) = T(4) + 1$$

The “+1” represents constant work done by the current call.

Derive the recurrence

Translate the trace into a statement that works for any n.

SMALLER PROBLEM

One call of size n−1

CURRENT WORK

Constant amount of work

BASE CASE

When n=1, stop directly

$$T(n) = T(n-1) + 1, \quad T(1) = 1$$

Recurrence = smaller-call cost + current-call cost.

Solve it by repeated expansion

Replace T(n−1) with what the recurrence says about T(n−1).

$$T(n) = T(n-1) + 1$$

$$= T(n-2) + 2$$

$$= T(n-3) + 3$$

$$\vdots$$

$$= T(n-k) + k$$

We are not guessing randomly — we are exposing the repeated pattern.

When does the expansion stop?

Stop when the remaining subproblem reaches the base case.

$$n - k = 1$$

$$k = n - 1$$

$$T(n) = T(1) + (n - 1)$$

$$T(n) \in \Theta(n)$$

Decrease by 1 → about n recursive levels → linear growth.

Pause & predict

Do not calculate fully. Use the call pattern.

RECURRENCE

$$T(n) = T(n-1) + 7$$
$$T(1)=1$$

QUESTION

Does changing +1 to +7 change the growth class?

A. $\Theta(1)$
B. $\Theta(\log n)$
C. $\Theta(n)$
D. $\Theta(n^2)$

Think about how many recursive levels occur, not the exact constant work per level.

Reveal: constants change the amount, not the growth shape

The recurrence still decreases by 1 each time.

$$T(n) = T(n-1) + 7$$

There are about n recursive levels, and each level does constant work.

$$T(n) = 7(n-1) + T(1) \in \Theta(n)$$

This is the recursive version of ignoring constant factors in asymptotic analysis.

Worked example 2: two recursive calls — Tower of Hanoi

Branching can matter more than how slowly n decreases.

PSEUDOCODE

```
HANOI(n)
    if n = 1
        move one disk
        return
    HANOI(n − 1)
    move largest disk
    HANOI(n − 1)
```

KEY DIFFERENCE

Each call creates TWO recursive subproblems of size n−1.

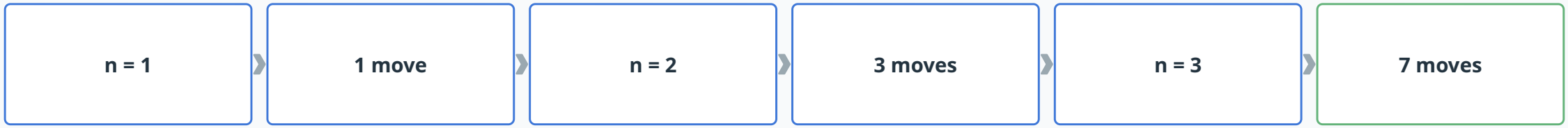
QUESTION

How quickly does the total number of moves grow?

This is our first branching recurrence: recursive work does not always form one chain.

Trace the work for small n

Count the required moves before generalizing.



n = 4 → 15 moves

$$1, 3, 7, 15, \ldots$$

Each extra disk roughly doubles the previous work and adds one more move.

Write the Tower of Hanoi recurrence

Two size-(n−1) recursive calls + one current move.

$$T(n) = 2T(n-1) + 1, \quad T(1)=1$$

$$= 2[2T(n-2)+1] + 1$$

$$= 4T(n-2) + 3$$

$$= 8T(n-3) + 7$$

Expansion reveals a powers-of-two pattern.

Tower of Hanoi: how the k-pattern appears

Write the first few expansions in the same form before generalizing.

$$k=1:\ T(n)=2T(n-1)+1$$

$$k=2:\ T(n)=2^2T(n-2)+(2^2-1)$$

$$k=3:\ T(n)=2^3T(n-3)+(2^3-1)$$

$$\text{after } k:\ T(n)=2^kT(n-k)+(2^k-1)$$

The coefficient of T doubles each time; the added constants are 1, 3, 7, … = $2^k - 1$.

Tower of Hanoi: substitute the stopping value

Now use the base case T(1)=1 instead of jumping directly from k=n−1 to Θ(2ⁿ).

$$n - k = 1 \quad \Rightarrow \quad k = n - 1$$

$$T(n) = 2^{n-1}T(1) + (2^{n-1} - 1)$$

$$= 2^{n-1} + 2^{n-1} - 1$$

$$= 2^n - 1 \quad \Rightarrow \quad T(n) \in \Theta(2^n)$$

For the classic Tower of Hanoi, this also gives the exact minimum number of moves: $2^n - 1$.

General pattern and stopping point

Keep expanding until the remaining subproblem reaches the base case.

$$T(n) = 2^k T(n-k) + (2^k - 1)$$

Stop when $n-k = 1$

$$k = n-1$$

$$T(n) \in \Theta(2^n)$$

Two almost-full-size recursive calls create exponential growth.

One call vs. two calls: a huge difference

Both reduce n by 1 — branching changes everything.

ONE RECURSIVE CALL

$T(n)=T(n-1)+1$

One chain of calls

Result: $\Theta(n)$

TWO RECURSIVE CALLS

$T(n)=2T(n-1)+1$

A branching call tree

Result: $\Theta(2^n)$

Always count BOTH the number of recursive calls and the size of each call.

Worked example 3: current work grows with n

One smaller call, but each level performs more work.

$$T(n) = T(n-1) + n$$

$$= T(n-2) + (n-1) + n$$

$$= T(n-3) + (n-2) + (n-1) + n$$

$$= T(1) + 2 + 3 + \cdots + n$$

Now the arithmetic sum — not branching — drives the growth.

Finish the arithmetic-series calculation

Taking T(1)=1 for a concrete calculation, simplify the sum all the way before stating Θ.

$$T(n) = T(1) + 2 + 3 + \cdots + n$$

$$= 1 + [\, n(n+1)/2 - 1 \,]$$

$$= n(n+1)/2$$

$$= \tfrac{1}{2}n^2 + \tfrac{1}{2}n \;\Rightarrow\; \Theta(n^2)$$

The quadratic term dominates; the linear term and constants do not change the tight asymptotic class.

The arithmetic series makes it quadratic

Connect recurrence analysis back to the summations from Lectures 6 and 7.

$$2 + 3 + \cdots + n = n(n+1)/2 - 1$$

$$= \tfrac{1}{2}n^2 + \tfrac{1}{2}n - 1$$

$$T(n) \in \Theta(n^2)$$

One recursive chain can still be quadratic when each level becomes more expensive.

PART B — LECTURE 9

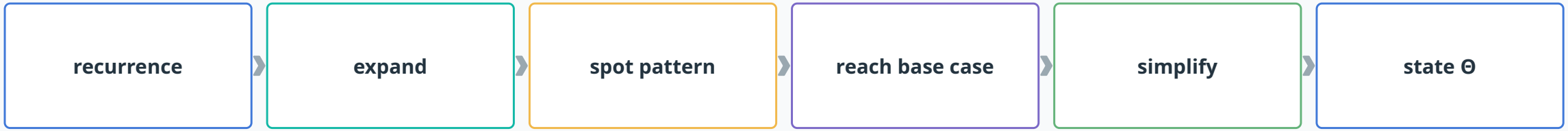
Iteration / expansion and substitution

Expansion reveals the pattern. Substitution turns a proposed asymptotic bound into a proof.

Build understanding from a concrete trace before generalizing the mathematics.

What repeated expansion is doing

It converts a recursive definition into ordinary arithmetic that we already know how to analyze.



Expansion is often the best first method when students are still learning what the recurrence means.

Worked example 4: uneven branching — naive Fibonacci

Recursive calls do not have to be equal-sized.

$$T(n) = T(n-1) + T(n-2) + 1$$

One call solves size $n-1$

A second call solves size $n-2$

The two branches overlap heavily

The same smaller problems are recomputed many times

This recurrence is irregular — it is not the balanced $aT(n/b)+f(n)$ form.

Where does φ ≈ 1.618 come from?

The +1 recurrence can be converted into the ordinary Fibonacci recurrence.

$$\text{Let } S(n) = T(n) + 1$$

$$S(n) = S(n-1) + S(n-2)$$

$$\text{Try } S(n)=r^n: \quad r^2 = r + 1$$

$$r = (1+\sqrt{5})/2 = \varphi \approx 1.618 \quad \Rightarrow \quad T(n)=\Theta(\varphi^n)$$

The exact φ derivation is optional; the main lesson is that repeated branching with overlap produces exponential growth.

Why naive Fibonacci grows so quickly

The call tree branches repeatedly and recomputes the same subproblems.

$$T(n)=T(n-1)+T(n-2)+1$$

$F(n-2), F(n-3), \ldots$ appear again and again

The number of calls grows exponentially

Runtime: exponential (in fact $\Theta(\varphi^n)$, where $\varphi \approx 1.618$)

Important lesson: recurrence trees can be unbalanced, and Master Theorem will not fit every recurrence.

Practice: derive a recurrence from code

Do not solve it yet. First model it correctly.

PSEUDOCODE

```
MYSTERY(n)
    if n <= 1
        return 1
    x = MYSTERY(n - 1)
    return x + 5
```

YOUR JOB

1. How many recursive calls?
2. What size?
3. How much current work?
4. What is the base case?

$$T(n) = ?$$

Model first. Solve second.

Answer: one n−1 call + constant work

The “+5” arithmetic is still constant-time in this model.

CALLS

1 recursive call

SIZE

n−1

OUTSIDE WORK

Θ(1)

$$T(n) = T(n-1) + \Theta(1)$$

$$\text{Therefore } T(n) \in \Theta(n)$$

The exact constant is less important than the number of recursive levels.

Practice: another code-to-recurrence translation

This time the algorithm creates THREE recursive calls.

PSEUDOCODE

```
TRIPLE(n)
    if n <= 1
        return
    TRIPLE(n / 3)
    TRIPLE(n / 3)
    TRIPLE(n / 3)
    doLinearWork(n)
```

THINK

Three size-n/3 calls contribute 3T(n/3).
The current call also does linear work.

$$T(n) = ?$$

Balanced divide-and-conquer does not have to mean “split into two halves.”

Answer: a three-way divide-and-conquer shape

Three one-third-size calls + linear work outside recursion.

$$T(n) = 3T(n/3) + n$$

3

number of recursive calls

n/3

size of each call

n

work outside recursion

This form will be useful when we generalize to aT(n/b)+f(n).

What is the substitution method?

In the formal version, we propose a bound and verify it using induction.

1 — GUESS

Based on expansion, a tree, or experience, propose $T(n)=O(g(n))$.

2 — ASSUME

Assume the bound holds for smaller inputs.

3 — VERIFY

Substitute the assumption into the recurrence and show the bound still holds.

Expansion helps you discover a plausible answer; substitution can prove it.

A light substitution proof: $T(n)=T(n-1)+n$

Expansion suggested $\Theta(n^2)$. Now verify an $O(n^2)$ upper bound.

GUESS / INDUCTION HYPOTHESIS

Suppose for every smaller m:

$T(m) \le c\, m^2$

for a suitable positive constant c.

SUBSTITUTE

$T(n)=T(n-1)+n$

$\le c(n-1)^2+n$

$= cn^2 + (-2c+1)n + c$

Choose $c=1$. Then the extra term is $-n+1 \le 0$ for $n \ge 1$, so $T(n) \le n^2$ (with the base case chosen consistently).

What did the substitution proof show?

The proposed quadratic upper bound survives the recursive step.

$$T(n) \in O(n^2)$$

Expansion gives $2+3+\cdots+n$, so $T(n) \in \Omega(n^2)$

Matching upper and lower bounds give:

$$T(n) \in \Theta(n^2)$$

Substitution is verification: first understand the recurrence, then prove the bound.

When should you use expansion vs. substitution?

They are related, but they serve slightly different purposes.

EXPANSION / ITERATION

Best when you want to SEE the repeated pattern.

- excellent for simple recurrences
- often gives an exact or nearly exact expression
- useful for forming a guess

SUBSTITUTION

Best when you already have a candidate asymptotic bound and want to VERIFY it.

- induction-based
- handles inequalities
- useful when exact expansion is messy

Next we need an algorithm rich enough to make both methods interesting: Merge Sort.

PART C — BRIDGE TO LECTURE 10

Meet Merge Sort before analyzing it

Students should understand the sorting algorithm before being asked to understand its recurrence.

Build understanding from a concrete trace before generalizing the mathematics.

What problem is Merge Sort solving?

Given n values, rearrange them into nondecreasing order.

INPUT

[8, 3, 6, 2, 7, 1, 5, 4]



OUTPUT

[1, 2, 3, 4, 5, 6, 7, 8]

This is our first sorting algorithm taught in detail.

A tiny preview of Divide and Conquer

Merge Sort follows three simple phases.

DIVIDE

Split the problem into smaller subproblems.

CONQUER

Solve the smaller problems recursively.

COMBINE

Merge the smaller sorted answers into one sorted result.

Full Divide-and-Conquer design comes later; here we need only enough to understand Merge Sort’s recurrence.

Merge Sort in one sentence

Split until every piece has one element, then merge sorted pieces back together.



The recursion happens while splitting; the important non-recursive work happens while merging.

Start with 8 values

We will trace the same array all the way down and back up.

[8, 3, 6, 2, 7, 1, 5, 4]

n = 8

Do not think about complexity yet. Just watch the algorithm.

Divide: split into two halves

Each half becomes a smaller sorting problem.

LEFT HALF

[8, 3, 6, 2]

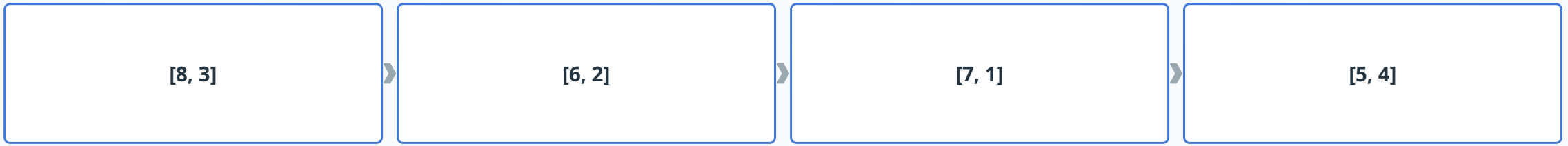
RIGHT HALF

[7, 1, 5, 4]

Two subproblems, each of size n/2.

Divide again

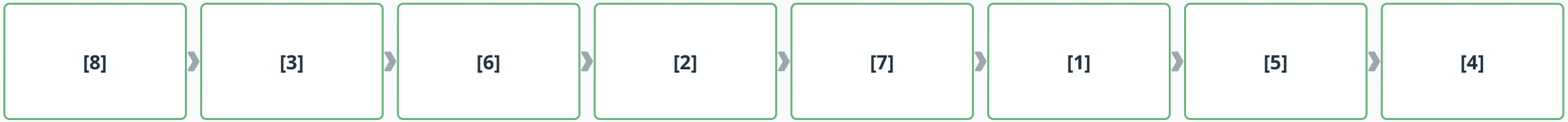
Each size-4 problem becomes two size-2 problems.



At this level there are four subproblems, each of size 2.

Base cases: single elements are already sorted

There is nothing left to divide.



The base case is size 1. Now the algorithm starts combining upward.

Combine: merge two sorted one-element lists

Always take the smaller front value first.

LEFT

[8]

RIGHT

[3]

COMPARE

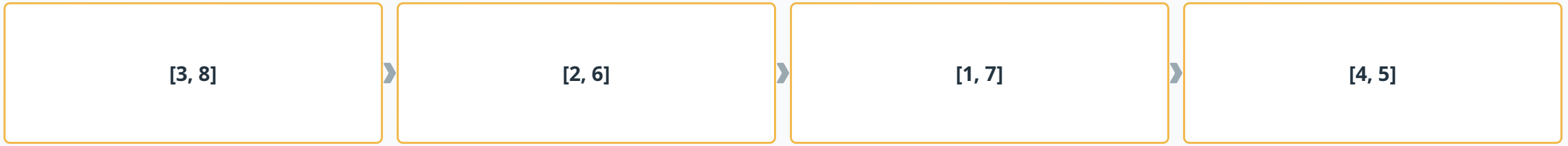
3 < 8 → output 3 first

MERGED → [3, 8]

Merging two lists with a total of m elements takes Θ(m) work.

Merge all size-1 pairs

Four sorted size-2 lists are produced.



The algorithm is now moving upward through the recursion.

Merge size-2 lists into size-4 lists

The same merging rule works at every scale.

LEFT SIDE

[3, 8] + [2, 6] → [2, 3, 6, 8]

RIGHT SIDE

[1, 7] + [4, 5] → [1, 4, 5, 7]

Every merge processes each element in its two input lists at most a constant number of times.

Final merge

Combine the two sorted halves into the final sorted array.

LEFT SORTED HALF

[2, 3, 6, 8]

RIGHT SORTED HALF

[1, 4, 5, 7]

[1, 2, 3, 4, 5, 6, 7, 8]

Now we understand the algorithm well enough to analyze its recursive cost.

What does one Merge Sort call do?

Separate recursive work from the work performed by the current call.

DIVIDE

Find the middle and split the range.

Cost: Θ(1) in the simplified model.

CONQUER

Sort left half recursively.
Sort right half recursively.

Cost: 2T(n/2)

COMBINE

Merge two sorted halves.
Every element participates in the merge.

Cost: Θ(n)

The recurrence comes directly from these three phases.

Derive the Merge Sort recurrence

Two half-size recursive calls + linear merging work.

$$T(n) = 2T(n/2) + n$$

2

two recursive calls

n/2

size of each call

+ n

merge work

Base case: $T(1)=\Theta(1)$.

Connect the recurrence to n = 8

Do not let the symbols become detached from the actual algorithm.

$$T(8) = 2T(4) + 8$$

$$T(4) = 2T(2) + 4$$

$$T(2) = 2T(1) + 2$$

$$T(1) = \text{constant}$$

The recurrence is just a compact description of the split/merge trace you already saw.

PART D — END OF LECTURE 9

Solve Merge Sort by expansion

Before drawing a recurrence tree, expose the same structure algebraically.

Build understanding from a concrete trace before generalizing the mathematics.

First expansion of Merge Sort

Replace T(n/2) using the same recurrence.

$$T(n) = 2T(n/2) + n$$

$$= 2[2T(n/4) + n/2] + n$$

$$= 4T(n/4) + n + n$$

$$= 4T(n/4) + 2n$$

Notice: the number of subproblems doubled, but the total merge work added another n.

Expand one more time

The same pattern repeats.

$$4T(n/4) + 2n$$

$$= 4[2T(n/8) + n/4] + 2n$$

$$= 8T(n/8) + n + 2n$$

$$= 8T(n/8) + 3n$$

After three levels, the outside work has accumulated to 3n.

Merge Sort: see the k-pattern before generalizing

Place the first expansions side by side, then write the formula for an arbitrary k.

1 expansion: $2T(n/2) + 1n$

2 expansions: $4T(n/4) + 2n$

3 expansions: $8T(n/8) + 3n$

k expansions: $2^kT(n/2^k) + kn$

Each expansion doubles the number of subproblems, halves their size, and adds one more n of merge work.

General pattern after k expansions

Look at what changes and what stays regular.

$$T(n) = 2^k T(n/2^k) + k\,n$$

2^k

number of subproblems

$n/2^k$

size of each subproblem

$k\,n$

accumulated merge work

Now use the base case to determine k.

Solve the Merge Sort stopping equation step by step

The base case is reached when each remaining subproblem has size 1.

$$n / 2^k = 1$$

$$n = 2^k$$

$$\log_2 n = \log_2(2^k) = k$$

$$k = \log_2 n \quad \text{and} \quad 2^k = n \quad \Rightarrow \quad n \text{ leaves}$$

This gives Merge Sort logarithmic recursion depth and n base-case leaves.

Stop when the Merge Sort subproblem size becomes 1

Use the base case to determine how many split levels occur.

$$n / 2^k = 1$$

$$k = \log_2 n$$

$$2^k = n$$

$$\#\ \text{leaves} = n$$

The recursion has $\log_2 n$ split levels and n base-case leaves.

Substitute k = $\log_2 n$ into the pattern

The algebra now matches the visual intuition we will use in Lecture 10.

$$T(n) = 2^k T(1) + kn$$

$$= nT(1) + n \log_2 n$$

$$= \Theta(n) + \Theta(n \log n)$$

$$T(n) \in \Theta(n \log n)$$

Merge Sort is faster-growing than linear, but dramatically better than quadratic for large n.

Why n log n makes intuitive sense

Each merge level processes all n elements, and the number of split levels is logarithmic.

WORK PER LEVEL

All merge operations on one level together touch about n elements.

NUMBER OF LEVELS

Repeated equal splitting creates about $\log_2 n$ levels.

Repeated halving creates about $\log_2 n$ levels.

$$n \text{ work/level} \times \log n \text{ levels} = n \log n$$

Lecture 10 will make this picture literal with a recurrence tree.

PART E — LECTURE 10

Recurrence Trees

Turn the expansion into a picture: nodes are subproblems, and each level shows how work is distributed.

Build understanding from a concrete trace before generalizing the mathematics.

Why use a recurrence tree?

Expansion can become messy. A tree shows the same recursive structure level by level.

NODE

Represents one recursive subproblem.

LEVEL

Contains all subproblems at the same recursion depth.

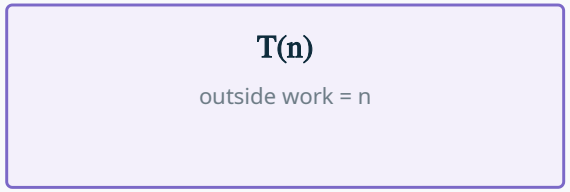
NODE COST

Represents work done outside recursive calls for that subproblem.

Add node costs across each level, then add the levels.

Merge Sort tree — level 0

Start with the original size-n problem.

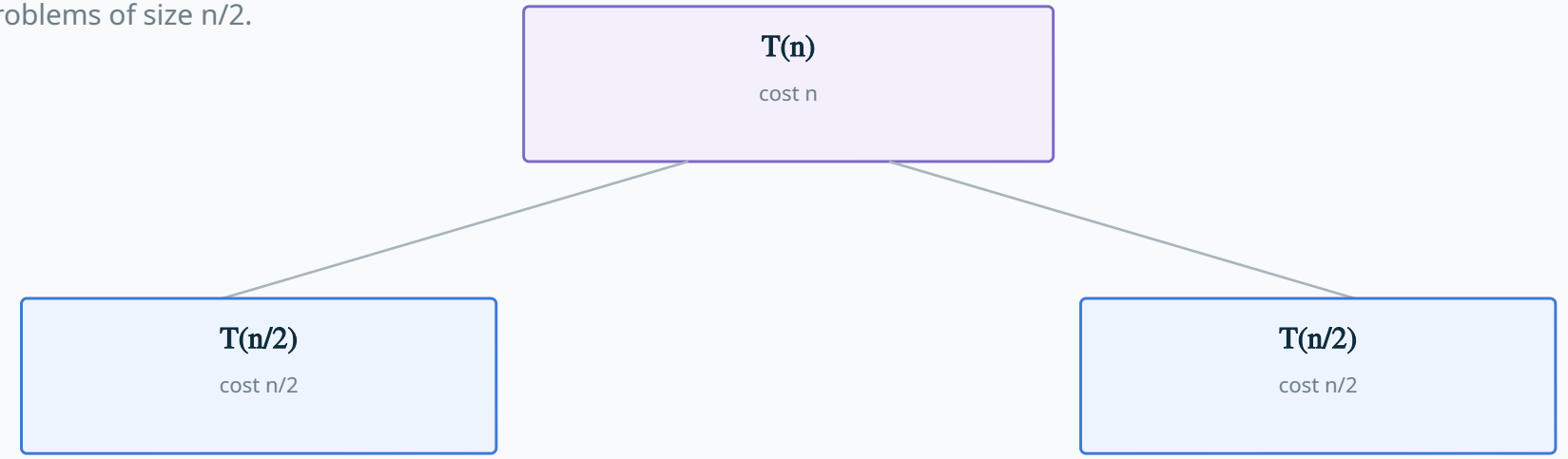


LEVEL 0 COST = n

The root does n merge work after its two recursive calls return.

Merge Sort tree — level 1

The root creates two subproblems of size n/2.

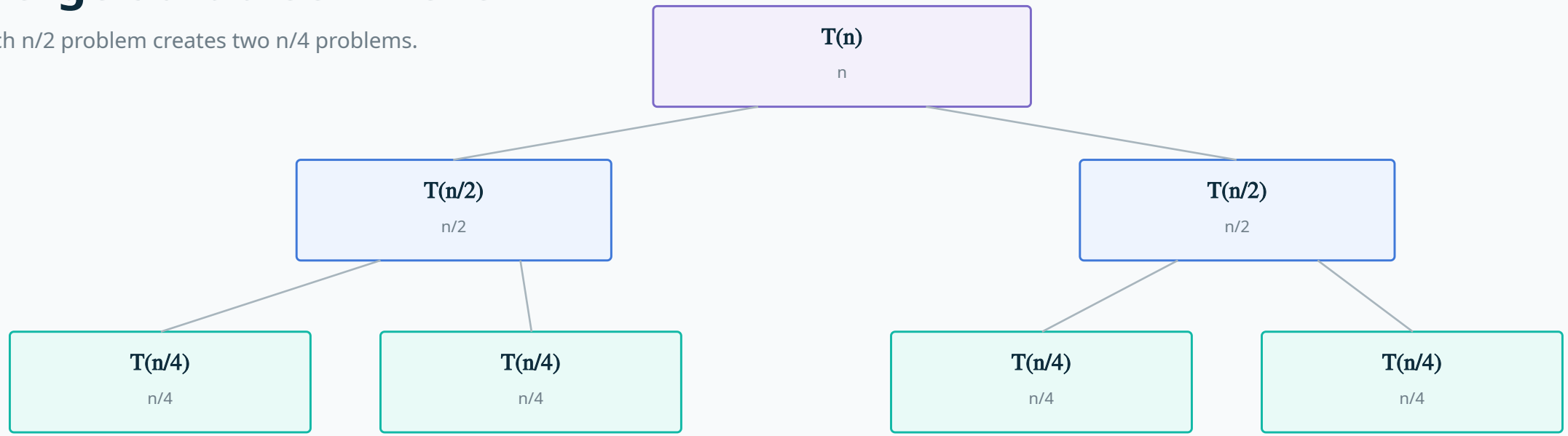


LEVEL 1 COST = n/2 + n/2 = n

The work is split across more nodes, but the total for the level remains n.

Merge Sort tree — level 2

Each n/2 problem creates two n/4 problems.



LEVEL 2 COST = 4 × (n/4) = n

Every level we have seen costs n.

General level i

Turn the visual pattern into a formula.

NUMBER OF NODES

2^i

SIZE OF EACH

$n / 2^i$

COST PER NODE

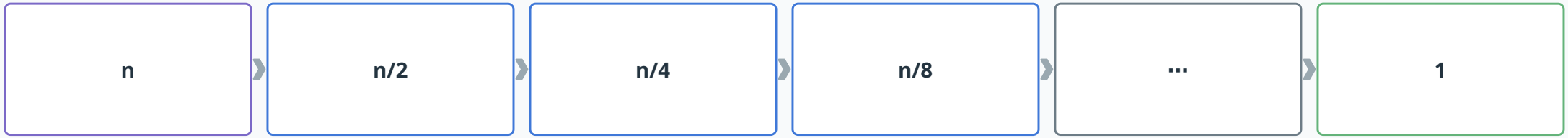
$n / 2^i$

$$\text{LEVEL COST} = 2^i \times (n / 2^i) = n$$

The number of nodes grows exactly as fast as each node becomes smaller.

How tall is the tree?

Follow one root-to-leaf path and keep halving.



$$n / 2^k = 1 \rightarrow k = \log_2 n$$

Tree height = Θ(log n).

What happens at the leaves?

At the bottom, every subproblem has size 1.

NUMBER OF LEAVES

$2^{(\log_2 n)} = n$

COST PER LEAF

$\Theta(1)$

TOTAL LEAF COST = $\Theta(n)$

The leaf cost is smaller than the n log n internal-level total, so it does not change the final class.

Add the levels

Each of about $\log_2 n$ internal levels costs n.

$$n + n + n + \cdots + n$$

about $\log_2 n$ times

$$\text{internal work} = n \log_2 n$$

$$+ \text{ leaf work } \Theta(n)$$

$$\text{TOTAL} = \Theta(n \log n)$$

Same answer as expansion — but now we can see why.

Same Merge Sort recurrence, two analysis methods

Different mathematics; same algorithm and same result.

EXPANSION / SUBSTITUTION

$T(n)=2T(n/2)+n$
→ $4T(n/4)+2n$
→ $8T(n/8)+3n$
→ pattern
→ $k=\log n$
→ $\Theta(n \log n)$

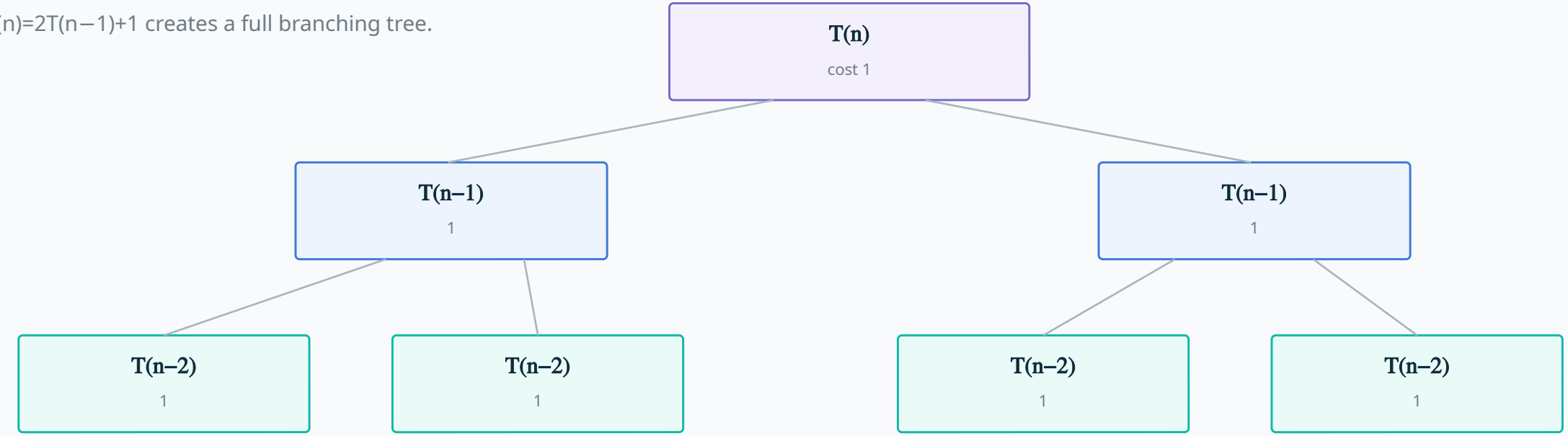
RECURRENCE TREE

level 0 cost = n
level 1 cost = n
level 2 cost = n
…
log n levels
→ $\Theta(n \log n)$

A good analyst can choose the representation that makes the recurrence easiest to understand.

Second tree example: Tower of Hanoi recurrence

T(n)=2T(n−1)+1 creates a full branching tree.



Level costs are 1, 2, 4, 8, … and the tree has about n levels.

Why 1 + 2 + 4 + ⋯ becomes $2^n - 1$

Use the standard geometric-series subtraction instead of jumping directly to the result.

$$S = 1 + 2 + 4 + \cdots + 2^{n-1}$$

$$2S = 2 + 4 + 8 + \cdots + 2^n$$

$$2S - S = 2^n - 1$$

$$S = 2^n - 1 \quad \Rightarrow \quad \Theta(2^n)$$

The number of nodes doubles every level, so the last level is the same order as the whole sum.

Total work for T(n)=2T(n−1)+1

Branching doubles the number of nodes at every level.

$$\text{level i cost} = 2^i$$

$$\text{last level} \approx 2^{n-1} \text{ nodes}$$

$$1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1$$

$$T(n) \in \Theta(2^n)$$

COUNTDOWN and HANOI both reduce n by 1 — only HANOI branches, so their growth is radically different.

Advanced tree example: T(n)=3T(n/3)+n

Now each call creates three one-third-size children.

LEVEL 0

1 node × n
= n

LEVEL 1

3 nodes × n/3
= n

LEVEL 2

9 nodes × n/9
= n

Every internal level still costs n.

Three-way split: solve the height and total work

Use the same stopping logic as Merge Sort, but replace 2-way halving with 3-way splitting.

$$n / 3^k = 1 \quad \Rightarrow \quad n = 3^k$$

$$k = \log_3 n$$

$$\text{internal work} = n + n + \cdots + n \quad (k \text{ levels})$$

$$= n \cdot \log_3 n \quad \Rightarrow \quad \Theta(n \log n)$$

The base of the logarithm changes only a constant factor: $\log_3 n = \log_2 n / \log_2 3$.

Three-way split: find height and total work

After i levels, each subproblem has size $n/3^i$.

$$n / 3^k = 1$$

$$k = \log_3 n$$

each internal level costs n

$$T(n) \in \Theta(n \log n)$$

Different branching factor, same level-sum idea: cost per level × number of levels.

How to choose an analysis method

Use the simplest method that makes the recurrence structure clear.

EXPANSION

Use when:
• recurrence is simple
• repeated substitution gives a clear pattern
• you want an intuitive derivation

SUBSTITUTION

Use when:
• you have a candidate bound
• you need to verify it formally
• inequalities are involved

RECURRENCE TREE

Use when:
• recursive branching matters
• work by level is easy to visualize
• you want to form a good bound guess

Lecture 11 adds a fourth option: Master Theorem, when the recurrence has the right form.

PART F — PRACTICE

Build the skill, not just the formula memory

First derive recurrences from code; then choose a solving method.

Build understanding from a concrete trace before generalizing the mathematics.

Practice A: derive the recurrence

Write T(n) first. Do not solve until the call structure is correct.

A

```
A(n):
 if n<=1 return
 A(n−1)
 constantWork()

T(n)=?
```

B

```
B(n):
 if n<=1 return
 B(n−1)
 B(n−1)
 constantWork()

T(n)=?
```

C

```
C(n):
 if n<=1 return
 C(n/3) three times
 linearWork(n)

T(n)=?
```

Identify number of calls, call sizes, outside work, and the base case.

Practice A — answers

Different recursive structures produce very different growth.

A

$T(n)=T(n-1)+1$

Result: $\Theta(n)$

B

$T(n)=2T(n-1)+1$

Result: $\Theta(2^n)$

C

$T(n)=3T(n/3)+n$

Result: $\Theta(n \log n)$

The recurrence structure — not one familiar keyword — determines the analysis.

Practice B: solve by expansion

Show at least two expansions before jumping to the final class.

1

$$T(n)=T(n-1)+3$$

2

$$T(n)=T(n-1)+n$$

3

$$T(n)=2T(n-1)+1$$

Expected answers: linear, quadratic, and exponential.

Practice B — solutions

Use the repeated pattern and a clear stopping condition.

1

$$T(n)=T(n-1)+3$$
$$=T(n-2)+6$$
$$=\dots$$
$$=T(1)+3(n-1)$$

$$\Theta(n)$$

2

$$T(n)=T(n-1)+n$$
$$=\dots$$
$$=T(1)+2+\dots+n$$

$$\Theta(n^2)$$

3

$$T(n)=2T(n-1)+1$$
$$=4T(n-2)+3$$
$$=\dots$$

$$\Theta(2^n)$$

Do not confuse recursion depth with total work when the call tree branches.

Practice C: recurrence-tree thinking

Compare two balanced trees with different branching factors.

TREE 1

$T(n)=2T(n/2)+n$

What is each level’s total cost?
How many levels?

TREE 2

$T(n)=3T(n/3)+n$

What is each level’s total cost?
How many levels?

Draw only the first 2–3 levels. Focus on total cost per level.

Practice C — answers

Different split factors can still produce the same asymptotic class.

TREE 1

2^i nodes × $n/2^i$ = n per level
height ≈ $\log_2 n$

TOTAL = Θ(n log n)

TREE 2

3^i nodes × $n/3^i$ = n per level
height ≈ $\log_3 n$

TOTAL = Θ(n log n)

The base of the logarithm changes only a constant factor, so both are Θ(n log n).

Recommended-book-aligned practice 1

Paraphrased from the style of CLRS Chapter 4 recurrence exercises.

EXERCISE

A recursive algorithm makes one call on input size $n-1$ and performs n units of additional work.

(a) Write the recurrence.
(b) Expand it to the base case.
(c) Give a tight asymptotic bound.

Try it before revealing the solution.

Solution 1

One size-(n−1) call + n outside work.

$$T(n)=T(n-1)+n$$

$$= T(n-2)+(n-1)+n$$

$$= T(1)+2+3+\cdots+n$$

$$\text{arithmetic sum} \rightarrow \Theta(n^2)$$

The recurrence is quadratic because the work accumulated along the single recursive chain keeps growing.

Recommended-book-aligned practice 2

A balanced divide-and-conquer example without a two-way split.

EXERCISE

An algorithm performs three recursive calls on input size n/3 and Θ(n) additional work.

(a) Write T(n).
(b) Draw the first three tree levels.
(c) Determine the total work.

Hint: calculate the TOTAL cost of each level, not only the cost of one node.

Solution 2

Three children per node, but each child handles only one third of the data.

$$T(n)=3T(n/3)+n$$

level costs: n, n, n, …

$$\text{height} = \log_3 n$$

$$n \times \log_3 n = \Theta(n \log n)$$

Changing the split from 2-way to 3-way changes constants and tree shape, but not this final Θ class.

Recommended-book-aligned practice 3: Merge Sort

Use the algorithm you now understand, not a memorized formula.

EXERCISE

For Merge Sort on n elements:

(a) Explain why there are two subproblems of size n/2.
(b) Explain why the combine step is Θ(n).
(c) Write the recurrence.
(d) Solve it using a recurrence tree.
(e) State the final tight bound.

The reasoning in (a) and (b) is more important than simply remembering the recurrence.

Solution 3: Merge Sort

Every term in the recurrence has an algorithmic meaning.

DERIVE

Two recursive halves:
$2T(n/2)$

Merge all n elements:
$+n$

$T(n)=2T(n/2)+n$

TREE

Each internal level costs n.

Height = $\log_2 n$.

Leaves add $\Theta(n)$.

RESULT

$n \times \log_2 n$
$+ \Theta(n)$

$T(n)=\Theta(n \log n)$

This is the core example connecting Lecture 9 and Lecture 10.

Common mistakes

Most errors happen before the algebra becomes difficult.

MISTAKE 1

Writing T(n)=T(n/2)+n for Merge Sort and forgetting the second recursive call.

MISTAKE 2

Looking only at how input size changes and ignoring how many recursive calls are created or how much work each call performs.

MISTAKE 3

Using substitution without first stating what bound is being proved.

MISTAKE 4

Drawing a recurrence tree but not summing node costs across each level.

MISTAKE 5

Memorizing Merge Sort = n log n without being able to explain where n and log n come from.

Derive first. Then solve. Then interpret.

A complete workflow for recursive analysis

Use the same checklist on every recursive algorithm.

1. **Define n**
2. **Identify recursive calls**
3. **Identify call sizes**
4. **Count outside work**
5. **Write base case**
6. **Write recurrence**
7. **Choose method**
8. **Solve**
9. **State Θ**
10. **Explain meaning**

This workflow matters more than memorizing isolated recurrence answers.

Lecture 9/10 recap

Five ideas to carry forward.

1. **A recurrence describes recursive running time using smaller input sizes.**
2. **Expansion exposes repeated structure and often reveals a pattern.**
3. **Substitution verifies a proposed asymptotic bound using induction.**
4. **A recurrence tree shows how work is distributed across recursion levels.**
5. **Merge Sort gives $T(n)=2T(n/2)+n$ and therefore $\Theta(n \log n)$.**

NEXT — LECTURE 11

Master Theorem

Can we recognize the recurrence-tree pattern without rebuilding the whole tree every time?

$$T(n) = aT(n/b) + f(n)$$

Recommended reading: CLRS Ch. 4 • Levitin Ch. 2